

Point sur les dépendances de tâches

Rappel sur l'ordonnancement des tâches

Par défaut, les tâches créées avec la directive `task` seront exécutées “à un moment”, par un thread quelconque, sans garantie sur leur ordre. Par conséquent, si ces tâches manipulent les mêmes valeurs, le résultat final de l'exécution du programme peut varier aléatoirement en fonction de l'ordre dans lequel les tâches ont été exécutées.

Présentation du principe de dépendance et utilisation basique

Exemple : modification puis somme de deux tableaux

Soient deux tableaux `a` et `b`. Dans un premier temps, on veut multiplier toutes les valeurs de `a` par 2, puis soustraire 5 à tous les éléments de `b`. Enfin, on additionne `a` et `b` dans `c`. Le code suivant montre une parallélisation naïve à base de tâches de cet algorithme:

```
int N = 10e6;
int* a, b, c;
// a[i] = 1
// b[i] = 10
init_array(a, b, c, N); //allocation mémoire et remplissage de a et b

for(long i = 0; i < N; i++)
{
    #pragma omp task // tâches 't_a'
    a[i] *= 2;
}

for(long i = 0; i < N; i++)
{
    #pragma omp task // tâches 't_b'
    b[i] -= 5;
}

for(long i = 0; i < N; i++)
{
    #pragma omp task // tâches 't_c'
    c[i] = a[i] + b[i];
}

#pragma omp taskwait
//on attend: c[i] = 2 + 5 = 7
```

Outre les performances au mieux médiocres de cette approche, que vous avez pu constater par vous-même au cours du TP, l'ordre aléatoire d'exécution des tâches fait que `c[i]` ne vaut pas tout le temps 7. Ci-dessous, des exemples de valeurs de `c[i]` selon l'ordre d'exécution des tâches pour un `i` donné :

```
t_a, t_b, t_c : 7
t_a, t_c, t_b : 12
t_b, t_c, t_a : 6
etc...
```

On peut donc se retrouver avec un tableau `c = [7, 7, 12, 6, 6, 12, ...]`

Cette implémentation parallèle n'est donc pas correcte : on ne retrouve pas le même résultat que pour la version séquentielle. Pourtant, ce programme est parallélisable, puisque :

- Le calcul de **a** et le calcul de **b** sont totalement indépendants et peuvent donc être effectués en même temps,
- Pour un **i** donné, on a juste besoin d'avoir déjà calculé **a[i]** et **b[i]** pour pouvoir calculer **c[i]**.

Comment tirer avantage de ces opportunités de parallélisation ? **En exprimant les dépendances entre les tâches.** Ici, en “langue naturelle”, on peut exprimer la dépendance en stipulant que “il faut calculer **a[i]** et **b[i]** avant de calculer **c[i]**”, ou “le calcul de **c[i]** a besoin de **a[i]** et **b[i]**”.

Graphe de dépendances

On peut faire un parallèle entre une tâche et une fonction : de la même façon qu'une fonction prend des arguments et renvoie un résultat, on peut définir les “arguments” (*input*) et des “valeurs de retour” (*output*) d'une tâche. Les “arguments” d'une tâche sont les valeurs auxquelles elle accède en lecture. Les “valeurs de retour” sont les valeurs auxquelles elle accède en écriture. Ainsi :

- **t_a** : prend **a[i]** en *input* et ressort **a[i]** en *output*.
- **t_b** : prend **b[i]** en *input* et ressort **b[i]** en *output*.
- **t_c** : prend **a[i]** et **b[i]** en *input* et ressort **c[i]** en *output*.

On a une *dépendance* à partir du moment où une tâche *dépend* de la complétion d'une autre tâche pour s'exécuter, sans quoi le programme ne donne pas le bon résultat. Cela correspond à une situation où une tâche prend en *input* l'*output* d'une autre tâche.

Ces dépendances peuvent être représentées sous forme d'un graphe. L'utilisation d'une variable en *input* correspond à une flèche entrante, et le fait d'avoir une variable comme *output* correspond à une flèche sortante.

Pour notre exemple, on obtient donc :

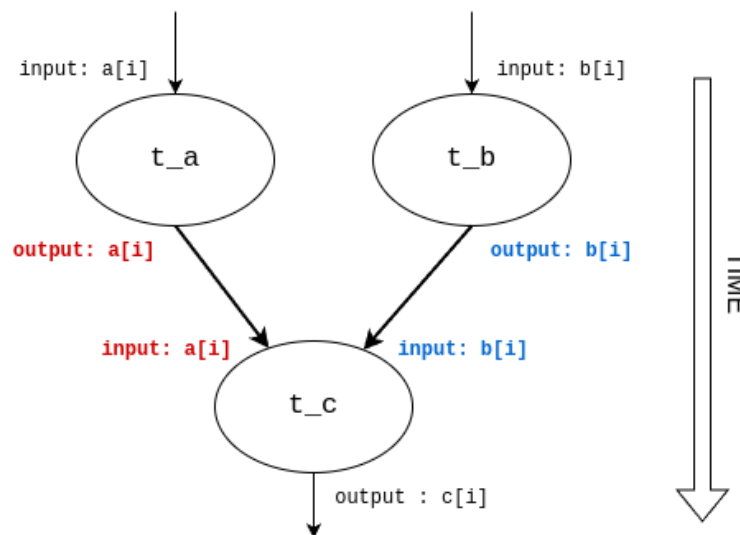


Figure 1: Inputs/outputs des tâches

Tous les *input/output* sont représentés, mais ceux qui induisent une *dépendance* entre tâches sont en gras et en couleur : ce sont ceux qui doivent être spécifiés lors de la création des tâches grâce à la clause **depend**.

Dans un graphe du genre, une dépendance correspond à **une flèche entre deux tâches** (ici, les flèches en gras). On omet en général les autres flèches.

Utiliser la clause `depend`

La clause `depend` se rajoute après `#pragma omp task` ainsi :

```
#pragma omp task depend(...)
{
    much_work(a, b);
}
```

Comme expliqué ci-dessus, une dépendance correspond à *une flèche entre deux tâches*, la partie “sortante” représentant le fait qu’une tâche “renvoie” (*output*) une variable, et la partie “entrante” représentant le fait qu’une tâche “prend en argument” (*input*) une variable.

Une clause `depend` correspond soit au début soit à la fin d’une flèche : il en faut donc deux pour définir une relation de dépendance.

On utilise `depend(out:A)` pour spécifier que la tâche a pour *output* la variable A, et `depend(in:A)` pour dire qu’elle prend A en *input*. En remplaçant dans le graphe précédent, on obtient :

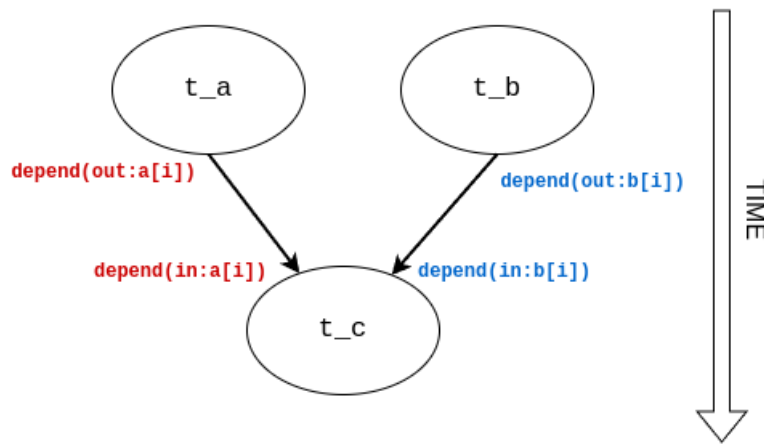


Figure 2: Correspondance avec les clauses `depend`

Ce qui, traduit en code dans notre exemple, donne :

```
int N = 10e6;
int* a, b, c;
// a[i] = 1
// b[i] = 10
init_array(a, b, c, N); //allocation mémoire et remplissage de a et b

for(long i = 0; i < N; i++){
    #pragma omp task depend(out:a[i]) // tâches 't_a'
    a[i] *= 2;
}

for(long i = 0; i < N; i++){
    #pragma omp task depend(out:b[i]) // tâches 't_b'
    b[i] -= 5;
}

for(long i = 0; i < N; i++){
    #pragma omp task depend(in:a[i],b[i]) // tâches 't_c'
```

```

        c[i] = a[i] + b[i];
    }

#pragma omp taskwait
//on attend: c[i] = 2 + 5 = 7

```

Le calcul de `c[i]` n'aura maintenant lieu qu'une fois que `a[i]` et `b[i]` ont été calculés.

Autres notes

Utilisation de plusieurs clauses `depend` et usage de `inout`

Si une tâche a à la fois des *input* et des *output*, elle portera plusieurs clauses `depend` :

```
#pragma omp task depend(in:A) depend(out:B)
```

Si elle dépend ou génère plusieurs ressources, celles-ci sont mises sous forme de liste dans une unique clause `depend` :

```
#pragma omp task depend(in:A,B) depend(out:C,D)
```

Si une ressource devrait apparaître à la fois dans une clause `in` et une clause `out`, on utilise à la place `inout`. Par exemple, on écrira pas `#pragma omp task depend(in:A, B) depend(out:B, C)` mais:

```
#pragma omp task depend(in:A) depend(out:C) depend(inout:B)
```

Transparence des variables utilisées dans les clauses `depend`

Dans l'exemple précédent, j'explique que le calcul de `c[i]` dépend de `a[i]` et `b[i]`, et utilise donc ces trois valeurs dans les clauses de dépendances. Cependant, le runtime OpenMP ne vérifie pas du tout que ces variables sont bien écrites/lues par les tâches. Il garantit simplement que les tâches ayant une clause `depend(in:TRUC)` ne seront pas lancées tant que les tâches `depend(out:TRUC)` n'auront pas fini de s'exécuter.

On peut donc remplacer `a[i]`, `b[i]` et `c[i]` par "n'importe quoi", tant que la logique du graphe ci-dessus est respectée. C'est à dire que le code suivant :

```

int a, b, c;

#pragma omp task depend(out:a)
a = compute_a();

#pragma omp task depend(out:b)
b = compute_b();

#pragma omp task depend(in:a, b)
c = compute_c(a, b);

```

aura exactement le même comportement que celui-ci :

```

int a, b, c;
int k, l;

#pragma omp task depend(out:k)
a = compute_a();

#pragma omp task depend(out:l)
b = compute_b();

#pragma omp task depend(in:k, l)
c = compute_c(a, b);

```

On pourrait exploiter ceci lorsqu'on découpe des domaines (blocs d'un tableau par exemple) pour faire du traitement par tâches. On pourrait alors numéroter les blocs pour exprimer les dépendances, même si le calcul se fait sur des éléments individuels du tableau.

Par exemple, reprenons notre exemple en faisant en sorte que chaque tâche traite $N/10$ éléments des tableaux:

```
int N = 10e6;
int elt = N/10;

int* a, b, c;
// a[i] = 1
// b[i] = 10
init_array(a, b, c, N); //allocation mémoire et remplissage de a et b

/* tableaux "dummy" pour numéroter les blocs et gérer les dépendances */
int blocks_a[10];
int blocks_b[10];

for(long i = 0; i < 10; i++){
    #pragma omp task depend(out:blocks_a[i]) // tâches 't_a'
    {
        for (long j = i * elt; j < (i+1) * elt ; j++)
            a[j] *= 2;
    }
}

for(long i = 0; i < 10; i++) {
    #pragma omp task depend(out:blocks_b[i]) // tâches 't_b'
    {
        for (long j = i * elt; j < (i+1) * elt; j++)
            b[i] -= 5;
    }
}

for(long i = 0; i < 10; i++){
    #pragma omp task depend(in:blocks_a[i],blocks_b[i]) // tâches 't_c'
    {
        for (long j = i * elt; j < (i+1) * elt; j++)
            c[i] = a[i] + b[i];
    }
}

#pragma omp taskwait
```

Ici, on traite chaque tableau par bloc de $N/10$ éléments. Il faut avoir computed tous les éléments du i ème bloc de **a** et de **b** avant de lancer la tâche calculant les éléments du i ème bloc de **c**.

Note : on pourrait aussi utiliser `depend(out:a[i*elt])` par exemple (le premier élément de chaque bloc).

Un exemple plus complet (factorisation LU) est disponible ici: <https://hpc2n.github.io/Task-based-parallelism/branch/spring2021/task-basics-lu/#id4>, avec de façon plus générale une bonne introduction à la programmation par tâches.

Bibliographie

Plus d'explications et d'exemples sur les graphes de dépendances de tâches : <https://hpc2n.github.io/Task-based-parallelism/branch/spring2021/motivation/>

Autres exemples simples d'utilisation des dépendances : https://passlab.github.io/Examples/contents/Chap__tasking/3_Task_Dependences.html

Une cheat-sheet des différentes clauses et directives liées aux tâches : <https://www.openmp.org/wp-content/uploads/OpenMPRef-5.0-1119-01-TSK-web.pdf>